INSTITUTO POLITÉCNICO NACIONAL
Escuela Superior de Cómputo (ESCOM)

Análisis y Diseño de Algoritmos

Práctica 2: Complejidad de ordenamiento

Alumnos:
Melo Cruz Leonardo Galdino
González Ramírez Ángeles Nataly

Grupo: 3BV1
Profesor: Andrés García Floriano

Repositorio: [https://github.com/natgor0305-hub/practica2-ordenamiento]

1. Introducción

El ordenamiento es uno de los problemas fundamentales del análisis de algoritmos: existen soluciones con complejidades muy distintas (cuadrática, casi lineal y lineal), y su comportamiento real depende también de la memoria disponible y de la jerarquía de caché. En esta práctica se implementaron cinco algoritmos de ordenamiento, se midió su tiempo de ejecución y la memoria consumida para arreglos de números reales de hasta mil millones de elementos, y se compararon los resultados experimentales con la complejidad teórica.

2. Objetivo

Implementar Bubble sort, Mergesort, Tree sort, Heapsort y Counting sort en dos lenguajes de programación (C++ y Python), evaluar su tiempo y memoria con arreglos de 10^5, 10^6, 10^7, 10^8 y 10^9 elementos, y analizar los resultados mediante tablas y gráficas.

3. Algoritmos implementados

Algoritmo	Mejor caso	Caso promedio	Peor caso	Memoria extra
Bubble sort (con bandera)	O(n)	$O(n^2)$	$O(n^2)$	O(1)
Mergesort	O(n log n)	O(n log n)	O(n log n)	O(n)
Tree sort (BST sin balancear)	O(n log n)	O(n log n)	$O(n^2)$	O(n)
Heapsort	O(n log n)	O(n log n)	O(n log n)	O(1)
Counting sort	O(n + k)	O(n + k)	O(n + k)	O(k)*

*Tabla 1. Complejidad teórica. k = tamaño del rango de llaves (10^6 en este experimento). *En nuestra implementación solo se guarda el arreglo de conteos.*

- **Bubble sort:** compara pares adyacentes e intercambia los desordenados; se detiene si una pasada no hace intercambios.
- **Mergesort:** divide el arreglo a la mitad, ordena recursivamente y mezcla usando un arreglo auxiliar del mismo tamaño.
- **Tree sort:** inserta todos los elementos en un árbol binario de búsqueda y los recupera con un recorrido inorden. La inserción y el recorrido son iterativos para evitar desbordar la pila con árboles profundos.
- **Heapsort:** construye un montículo máximo en el propio arreglo y extrae repetidamente la raíz; no requiere memoria adicional.
- **Counting sort:** cuenta las apariciones de cada llave y reconstruye el arreglo. Para aplicarlo a reales, los datos se generaron con dos decimales en [0, 9,999.99] y la llave entera es round(x · 100), con k = 10^6 llaves posibles.

4. Metodología

4.1 Entorno de pruebas

- Procesador: Intel Xeon @ 2.10 GHz, 1 núcleo asignado.
- Memoria RAM: 4 GB (≈ 3.9 GB disponibles).

- C++: g++ con -O2 y estándar C++17. Python: CPython 3 sin bibliotecas externas de ordenamiento (todos los algoritmos son implementación propia).

4.2 Datos y mediciones

- Los arreglos de números reales (double / float) se generan de forma pseudoaleatoria con semilla fija (42), por lo que todos los algoritmos ordenan exactamente los mismos datos.
- Tiempo: solo se mide la fase de ordenamiento (no la generación de datos), con reloj monótono (std::chrono::steady_clock en C++ y time.perf_counter en Python).
- Memoria: pico de memoria residente del proceso (ru_maxrss), que incluye el arreglo de entrada. Cada experimento corre en un proceso independiente.
- Tras cada ejecución se verifica que el arreglo quedó ordenado (columna "ordenado" de results.csv; todas las ejecuciones completadas resultaron correctas).
- Límite de 90 segundos por ejecución. Si se excede, se reporta "> 90 s" y se omiten los tamaños mayores. Si la memoria estimada excede el 90 % de la RAM disponible, no se ejecuta y se reporta "Sin RAM".

5. Resultados

5.1 C++

Tabla 2. Tiempo de ejecución en C++ (segundos).

Algoritmo	100,000	1,000,000	10,000,000	100,000,000	1,000,000,000
Bubble sort	20.00	> 90 s	> 90 s	> 90 s	> 90 s
Mergesort	0.0106	0.1277	1.76	19.62	Sin RAM
Tree sort	0.0199	0.5066	17.12	> 90 s	> 90 s
Heapsort	0.0159	0.2172	2.90	45.86	Sin RAM
Counting sort	0.0054	0.0167	0.0856	0.8816	Sin RAM

Tabla 3. Memoria pico en C++ (MB).

Algoritmo	100,000	1,000,000	10,000,000	100,000,000	1,000,000,000
Bubble sort	11.1	> 90 s	> 90 s	> 90 s	> 90 s
Mergesort	11.1	18.4	155.7	1529.0	Sin RAM
Tree sort	11.1	26.0	232.0	> 90 s	> 90 s
Heapsort	11.1	11.1	79.4	766.2	Sin RAM
Counting sort	11.1	14.6	83.3	769.9	Sin RAM

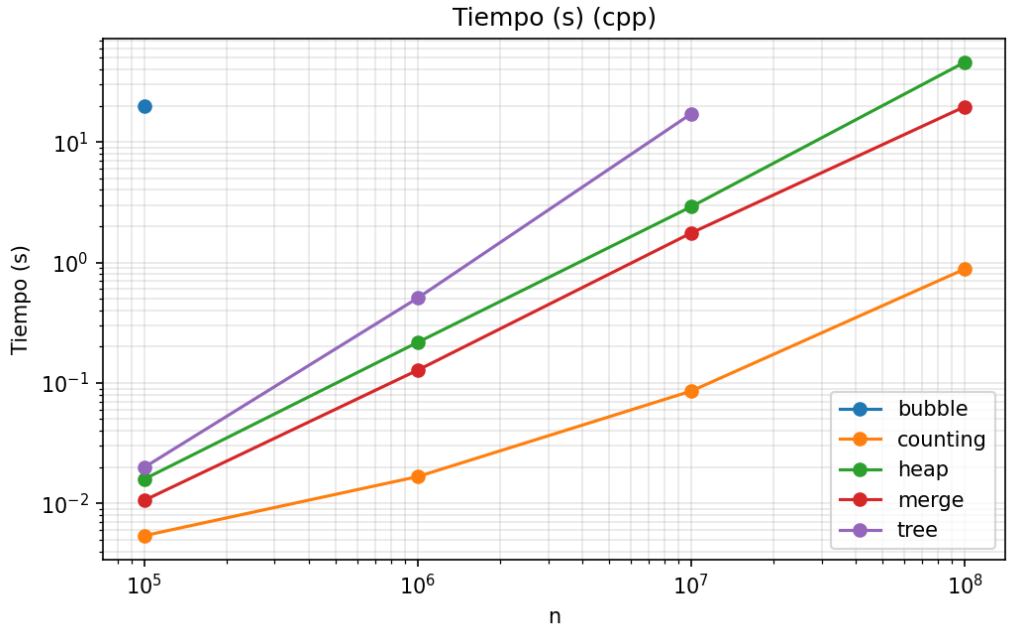


Figura 1. Tiempo de ejecución en C++ (escala logarítmica en ambos ejes).

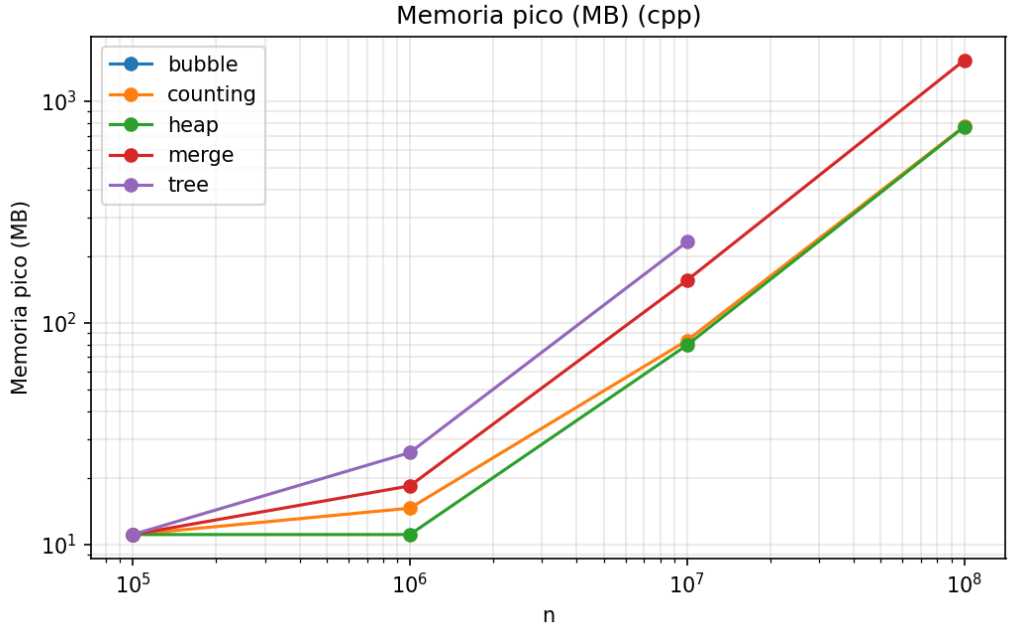


Figura 2. Memoria pico en C++ (escala logarítmica en ambos ejes).

5.2 Python

Tabla 4. Tiempo de ejecución en Python (segundos).

Algoritmo	100,000	1,000,000	10,000,000	100,000,000	1,000,000,000
Bubble sort	> 90 s	> 90 s	> 90 s	> 90 s	> 90 s
Mergesort	0.1719	2.53	36.74	Sin RAM	Sin RAM

Algoritmo	100,000	1,000,000	10,000,000	100,000,000	1,000,000,000
Tree sort	0.1403	3.36	81.57	Sin RAM	Sin RAM
Heapsort	0.2077	3.67	62.07	Error (mem.)	Error (mem.)
Counting sort	0.0622	0.3142	2.87	Error (mem.)	Error (mem.)

Tabla 5. Memoria pico en Python (MB).

Algoritmo	100,000	1,000,000	10,000,000	100,000,000	1,000,000,000
Bubble sort	> 90 s	> 90 s	> 90 s	> 90 s	> 90 s
Mergesort	14.5	55.8	468.8	Sin RAM	Sin RAM
Tree sort	19.9	109.4	1004.1	Sin RAM	Sin RAM
Heapsort	13.7	48.1	392.6	Error (mem.)	Error (mem.)
Counting sort	24.9	82.7	507.2	Error (mem.)	Error (mem.)

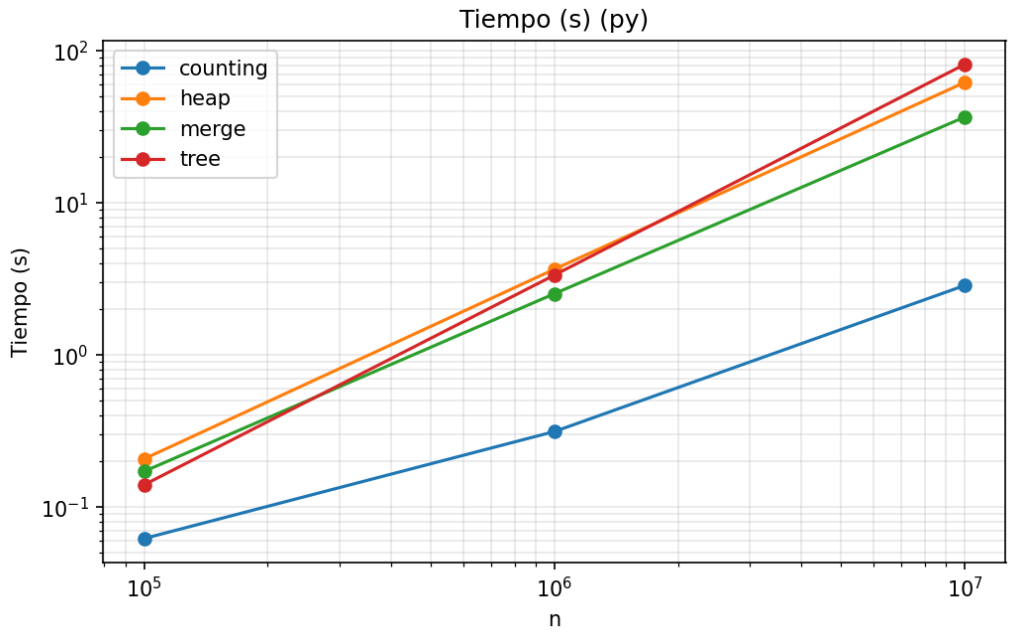


Figura 3. Tiempo de ejecución en Python (escala logarítmica en ambos ejes).

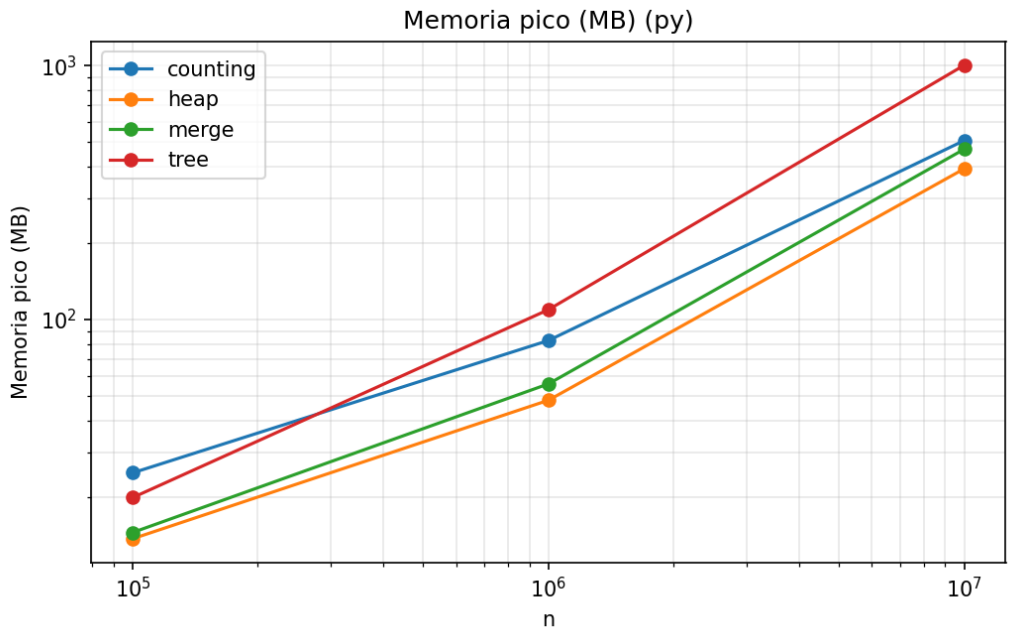


Figura 4. Memoria pico en Python (escala logarítmica en ambos ejes).

Leyenda de celdas sin dato: "> 90 s" = superó el límite de tiempo; "Sin RAM" = la memoria necesaria excede la disponible en el equipo; "Error (mem.)" = el proceso terminó con error, muy probablemente por falta de memoria (una lista de 10^8 floats en Python requiere más de 3 GB solo para almacenar los datos).

6. Análisis de resultados

6.1 Bubble sort: crecimiento cuadrático

Bubble sort tardó 20.00 s con 10^5 elementos en C++ y no terminó dentro del límite con 10^6. Como su complejidad es O(n^2), cada vez que n se multiplica por 10 el tiempo se multiplica por unas 100. Extrapolando a partir de la medición de 10^5, se estimarían unos 33 minutos para 10^6, cerca de 56 horas para 10^7, alrededor de 231 días para 10^8 y aproximadamente 63 años para 10^9. Es decir, el algoritmo es inviable en la práctica para los tamaños grandes, aunque es el único que no necesita memoria adicional significativa. En Python, Bubble sort ni siquiera terminó con 10^5 elementos en 90 s.

6.2 Algoritmos O(n log n): Mergesort, Heapsort y Tree sort

Mergesort y Heapsort crecen ligeramente más rápido que linealmente, como predice O(n log n): en C++, Mergesort pasó de 0.1277 s (10^6) a 1.76 s (10^7) y 19.62 s (10^8). Heapsort fue más lento que Mergesort en todos los tamaños y la brecha creció: con 10^8 elementos tardó 45.86 s, más del doble que Mergesort (19.62 s). Aunque ambos son O(n log n), Heapsort accede a posiciones muy separadas del arreglo (padre e hijos) y produce muchos fallos de caché cuando los datos no caben en ella, mientras que Mergesort recorre la memoria de forma secuencial.

Tree sort fue el más afectado por la memoria: de 10^6 a 10^7 elementos su tiempo se multiplicó por casi 34 (0.51 s a 17.12 s) en lugar de las ~11.5 veces esperadas, y con 10^8 superó los 90 s. Cada inserción recorre nodos dispersos en memoria, lo que genera muchos fallos de caché, además de que cada nodo ocupa 16 bytes y el árbol no está balanceado. Con datos aleatorios la profundidad promedio es logarítmica, pero con datos ya ordenados degeneraría en $O(n^2)$.

6.3 Counting sort: complejidad lineal

Counting sort fue el algoritmo más rápido en todos los tamaños: 0.0054 s con 10^5 y 0.88 s con 10^8 elementos en C++. De 10^7 a 10^8 el tiempo se multiplicó por ≈ 10.3 (crecimiento lineal), consistente con O(n + k) con k = 10^6 fijo; con n pequeño, el costo fijo de recorrer los k conteos suaviza el crecimiento. Su ventaja depende de que el rango de llaves sea pequeño respecto a n; si k fuera mucho mayor que n, el arreglo de conteos dominaría el tiempo y la memoria. Además, aquí se aprovechó que los reales tienen precisión fija de dos decimales; para reales arbitrarios no es aplicable directamente.

6.4 Consumo de memoria

Con 10^8 elementos en C++, el arreglo de entrada ocupa 800 MB. Heapsort y Counting sort usaron ≈ 770 MB (sin memoria adicional significativa; Counting sort solo añade 4 MB de conteos), Mergesort ≈ 1,529 MB (el doble, por el arreglo auxiliar) y Tree sort necesitó ≈ 232 MB con solo 10^7 elementos, es decir, unos 24 bytes por elemento (nodo de 16 bytes más el arreglo y la pila del recorrido), por lo que con 10^8 requeriría más de 2.4 GB. Con 10^9 elementos, incluso el arreglo de entrada (8 GB) excede la RAM del equipo de pruebas (4 GB), por lo que ningún algoritmo pudo ejecutarse con ese tamaño.

6.5 Comparación C++ vs. Python

Con 10^7 elementos, Python fue entre 5 y 34 veces más lento que C++: Mergesort 36.74 s vs. 1.76 s (≈ 21×), Heapsort 62.07 s vs. 2.90 s (≈ 21×), Counting sort 2.87 s vs. 0.086 s (≈ 34×) y Tree sort 81.57 s vs. 17.12 s (≈ 5×; en C++ el cuello de botella ya es la memoria, por lo que la brecha se reduce). Python también consume más memoria por elemento porque cada número es un objeto: con 10^7 elementos, Mergesort usó 469 MB frente a 156 MB en C++, y Tree sort 1,004 MB frente a 232 MB. Como consecuencia, en Python solo fue posible llegar hasta 10^7 elementos en la mayoría de los algoritmos.

7. Conclusiones

- Los resultados experimentales confirman la teoría: Bubble sort crece de forma cuadrática y se vuelve inviable; Mergesort y Heapsort se comportan como O(n log n); y Counting sort crece linealmente.
- La complejidad asintótica no cuenta toda la historia: Heapsort y Tree sort, con el mismo O(n log n) promedio que Mergesort, fueron notablemente más lentos con datos grandes por su patrón de acceso a memoria (caché).
- Hay un compromiso entre tiempo y memoria: Mergesort es rápido pero duplica la memoria; Heapsort no usa memoria extra pero es más lento; Counting sort es el más rápido, pero solo aplica si las llaves son enteros en un rango acotado.

- Con 10^9 elementos el factor limitante es la memoria RAM, no solo el tiempo: procesar ese tamaño requiere equipos con decenas de GB o algoritmos externos (ordenamiento en disco).
- La elección del lenguaje importa: la implementación en C++ fue de 5 a 34 veces más rápida y usó mucha menos memoria que la de Python para el mismo algoritmo.

8. Repositorio

El código fuente (sorting.cpp, sorting.py), los scripts de experimentación (run_benchmarks.py, make_report.py), los resultados (results.csv) y las gráficas están disponibles en: [https://github.com/natgor0305-hub/practica2-ordenamiento].